

2057 Demographic Methods - Practical 2 (Mortality and Standardisation)

2026-10-06

Synopsis

In this practical, you will learn how to retrieve data from an online repository (GitHub) to estimate crude death rates, age-specific mortality rates, and standardised death rates. These calculations will be used to answer a number of questions about direct and indirect standardisation. Building your R programming skills, this activity will demonstrate how to work with long- and wide-format data and how to present your results using basic plots.

The heading of the R Script

The R Script starts by clearing all data if any and installing and loading required packages (in this example, lines are commented out using hashtags to prevent execution). This practical only requires the package **ggplot2**, to draw plots.

```
rm(list = ls())  
#install.packages("ggplot2")  
library(ggplot2)
```

We can declare the URL from where the data will be downloaded, a GitHub repository in this case; and we can pull the data using the function **read.csv()**.

```
GitHub = "https://raw.githubusercontent.com/Romero-Prieto/teaching/main/Demographic%20Methods/pr  
data = read.csv(GitHub)
```

Data inspection

We have already used the function **class(object)** to determine the class of an object. The class of an object defines the different properties, associated methods, and programming rules. Knowing the class of data, we can inspect their content using the function **print(object)**, as shown below.

```
class(data)
```

```
## [1] "data.frame"
```

```
print(data)
```

```
##      x  n      Country Population Deaths  
## 1    0  1      Sweden      59727      279  
## 2    1  4      Sweden     229775       42  
## 3    5  5      Sweden     245172       31  
## 4   10  5      Sweden     240110       33  
## 5   15  5      Sweden     264957       61  
## 6   20  5      Sweden     287176       87  
## 7   25  5      Sweden     311111       98  
## 8   30  5      Sweden     280991      140  
## 9   35  5      Sweden     286899      197  
## 10  40  5      Sweden     308238      362  
## 11  45  5      Sweden     320172      643  
## 12  50  5      Sweden     242230      738  
## 13  55  5      Sweden     210785      972  
## 14  60  5      Sweden     216058     1640  
## 15  65  5      Sweden     224479     2752  
## 16  70  5      Sweden     222578     4509  
## 17  75  5      Sweden     184102     6745  
## 18  80  5      Sweden     140667     9587  
## 19  85 NA      Sweden     110242    17340
```

```
## 20 0 1 Kazakhstan 174078 3720
## 21 1 4 Kazakhstan 754758 1220
## 22 5 5 Kazakhstan 879129 396
## 23 10 5 Kazakhstan 808510 298
## 24 15 5 Kazakhstan 720161 561
## 25 20 5 Kazakhstan 622988 673
## 26 25 5 Kazakhstan 733057 752
## 27 30 5 Kazakhstan 732312 965
## 28 35 5 Kazakhstan 612825 1113
## 29 40 5 Kazakhstan 487996 1405
## 30 45 5 Kazakhstan 284799 1226
## 31 50 5 Kazakhstan 503608 2878
## 32 55 5 Kazakhstan 301879 3266
## 33 60 5 Kazakhstan 374317 5212
## 34 65 5 Kazakhstan 256247 6866
## 35 70 5 Kazakhstan 154623 6182
## 36 75 5 Kazakhstan 149917 8199
## 37 80 5 Kazakhstan 88716 9013
## 38 85 NA Kazakhstan 58940 10627
```

As described, we are working with a **data frame** consisting of five columns: x (a convention to denote the beginning of an age interval), n (the length of the age interval), *Country*, *Population* (mid-year population), and *Deaths* (during the same year). This is an example of a **long-form data**, which is an efficient arrangement to represent multiple variables (i.e., mid-year population and the number of deaths) for multiple analytical units (countries, in this practical).

Note that age intervals are typically 5 years long, except for the first age group [0,1), the second age group [1,5), and the final open-ended age group (starting at age 85, with its expected length to be determined).

Data preparation

We can create a **character** variable listing the names of the countries using the function `unique(object)`, keeping the first occurrence of each value and removing subsequent duplicates, and the function `sort(object)` to sort the list of countries in alphabetical order, as shown below. This list can be used for labels, tables, and plots.

```
countries = sort(unique(data[, "Country"]))
countries
```

```
## [1] "Kazakhstan" "Sweden"
```

We can create a **numeric** variable informing the number of age intervals using the function `length(object)`, as shown below. This number can be used for calculations.

```
ages = length(unique(data[, "x"]))
ages
```

```
## [1] 19
```

We can calculate age-specific mortality rates by dividing the number of deaths by the mid-year population for the same calendar year, and we can add this information to the data frame as a new column, as shown.

```
data[, "nMx"] = data[, "Deaths"]/data[, "Population"]
```

As a convention, we plot interval data at the midpoint of that interval (e.g., at exact age 7.5 for a rate between 5 and 10, or at exact time 2001.5 for a rate between 01/01/2001 and 01/01/2002). Hence, another data preparation is to add a new column to the data frame **age**, containing midpoint ages, as follows:

```
data[, "age"] = data[, "x"] + data[, "n"]/2
tail(data, 5)
```

```
##      x  n Country Population Deaths      nMx age
## 34 65  5 Kazakhstan  256247   6866 0.02679446 67.5
## 35 70  5 Kazakhstan  154623   6182 0.03998112 72.5
## 36 75  5 Kazakhstan  149917   8199 0.05469026 77.5
## 37 80  5 Kazakhstan   88716   9013 0.10159385 82.5
## 38 85 NA Kazakhstan   58940  10627 0.18030200  NA
```

For open-ended age intervals, we can use the reciprocal of the age-specific mortality rate as an estimate of the average length of that interval. Intuitively, the higher the mortality rate is in the open-ended age interval, the shorter the expected length of life will be. We will formalise this idea during the next two weeks, which will be dedicated to life table construction. Hence, we first identify the rows of the data frame corresponding to the open-ended age intervals (using a logical array *sEL*), and then we replace the midpoint age with *x* plus the reciprocal of the age-specific mortality rate, as follows:

```
sEL = (data[, "x"] == data[ages, "x"])
data[sEL, "age"] = data[sEL, "x"] + 1/data[sEL, "nMx"]
data[sEL, ]
```

```
##      x  n   Country Population Deaths      nMx      age
## 19 85 NA      Sweden   110242  17340 0.1572903 91.35767
## 38 85 NA Kazakhstan    58940  10627 0.1803020 90.54625
```

Demographic analysis

We have discussed the advantages of long-form data for data storage, efficient plotting, and the analysis of populations that share common characteristics while remaining independent of one another. However, wide-form data provide a convenient structure when these characteristics are expected to interact, as in the case of standardisation. Hence, we can use the function **unstack(object, value variable ~ group variable)** to create three independent data frames: ${}_nN_x$ (mid-year population), ${}_nD_x$ (number of deaths), and ${}_nM_x$ (age-specific mortality rates). In each data frame, the columns represent countries (in alphabetical order) and the rows represent age intervals, providing a typical example of **wide-form data**.

```
nNx = unstack(data, Population ~ Country)
nDx = unstack(data, Deaths ~ Country)
nMx = unstack(data, nMx ~ Country)
```

Then, we can then calculate the age-specific weights, *W*, for each country by dividing the age-specific population by the total population of that country, as shown below:

```
W = sweep(nNx, 2, colSums(nNx), "/")
```

Note that we cannot simply divide ${}_nN_x$ by the total sum of each column as **nNx/colSums(nNx)**. Instead, we can use the function **sweep(object, margin, vector, function)** to apply a function (margin = 2) of ${}_nN_x$, using the vector of total populations for each country. We can inspect the weights by printing the entire matrix and checking that the weights add up to 1 using the following two command lines:

```
print(W)
```

```
##      Kazakhstan      Sweden
## 1  0.02001159 0.01361930
## 2  0.08676516 0.05239462
## 3  0.10106255 0.05590554
## 4  0.09294436 0.05475127
## 5  0.08278797 0.06041703
## 6  0.07161720 0.06548353
## 7  0.08427047 0.07094133
## 8  0.08418482 0.06407319
## 9  0.07044889 0.06542037
## 10 0.05609885 0.07028621
## 11 0.03273981 0.07300747
## 12 0.05789356 0.05523469
## 13 0.03470328 0.04806441
## 14 0.04303058 0.04926679
## 15 0.02945754 0.05118700
## 16 0.01777509 0.05075352
## 17 0.01723410 0.04198000
## 18 0.01019858 0.03207570
## 19 0.00677560 0.02513802
```

```
colSums(W)
```

```
## Kazakhstan      Sweden
##           1           1
```

Exercise 1: Age-specific mortality rates and direct age-standardisation

a. Compute age-specific death rates for both countries and plot those on one graph. Discuss the differences between the two countries.

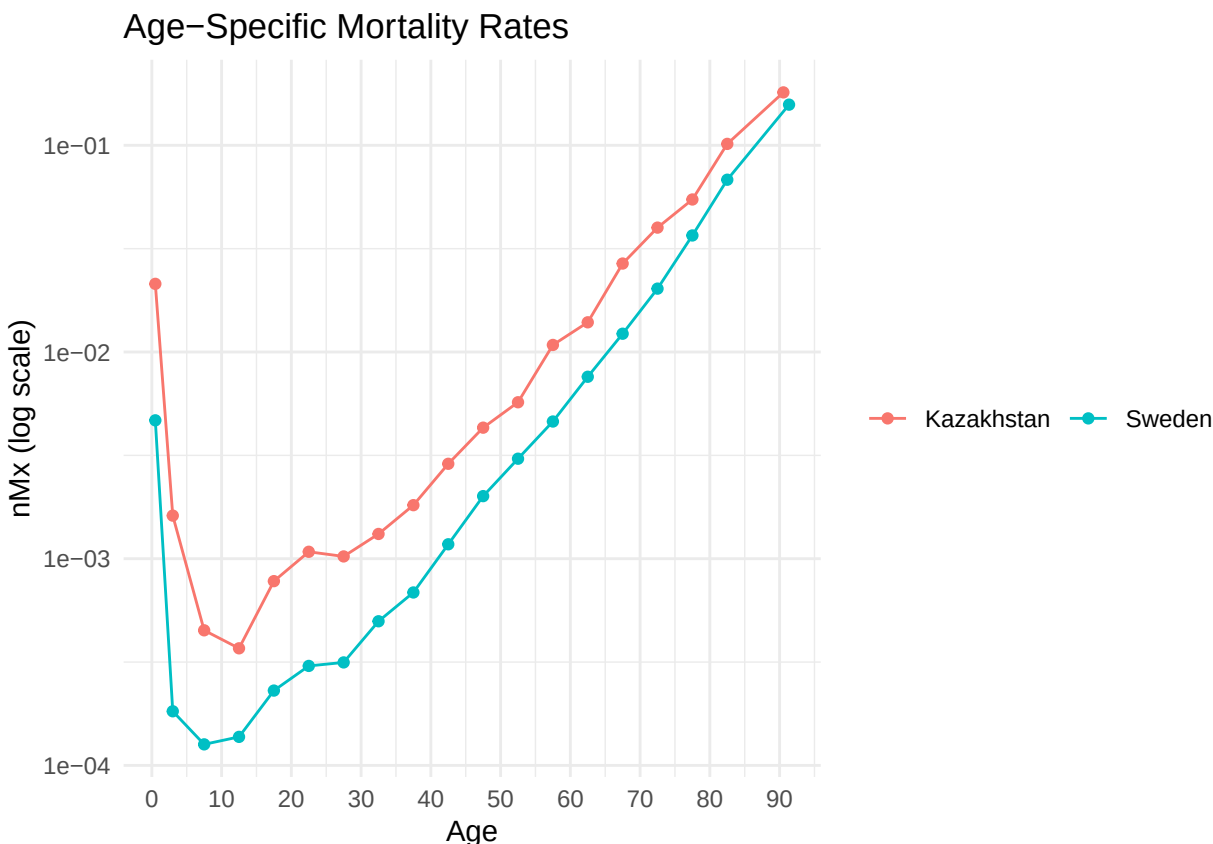
Hint: Age-specific mortality rates can be calculated directly by dividing the number of deaths by the mid-year population for the same calendar year. R does not require any special notation for this calculation.

$nMx = nDx / nNx$

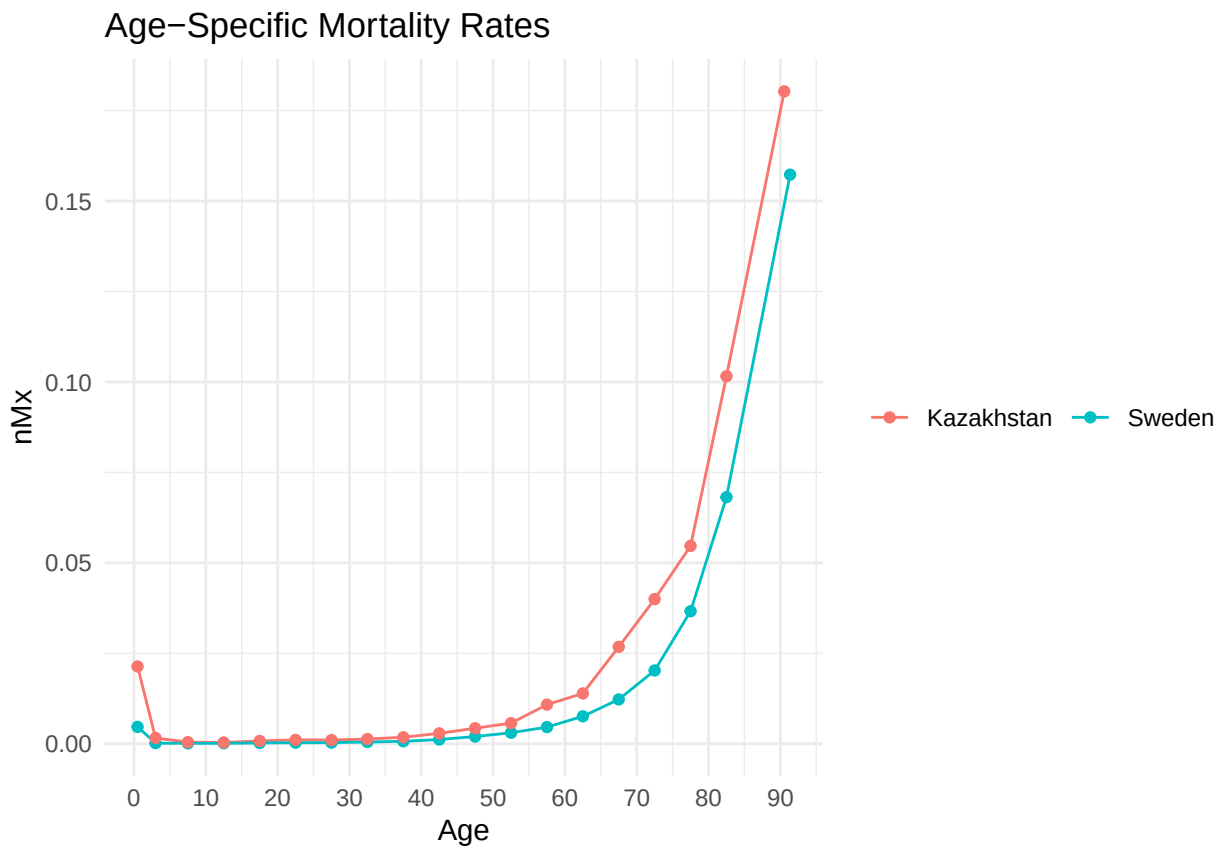
Hint for plotting: the function `ggplot()` has a special structure to control the features of a plot. The basic structure is `ggplot(name of the database, aes(x = name of the variable, x-axis, y = name of the variable, y-axis, color = name of the variable defining the groups of the plot, group = name of the variable defining the groups of the plot)) + geom_line() + geom_point()`. Use the command `help("ggplot")` to identify some other inputs that will control the features of a plot such as the scale, labels, etc.

If plotting a figure becomes challenging or time-consuming, you can either skip this part—to be discussed later during the solution—or ask Copilot to provide a code example.

```
lnM = ggplot(data, aes(x = age, y = nMx, color = Country, group = Country)) +  
  geom_line() + geom_point() +  
  labs(title = "Age-Specific Mortality Rates", x = "Age", y = "nMx (log scale)") +  
  theme_minimal() +  
  theme(legend.position = "right", legend.direction = "horizontal", legend.title = element_blank()) +  
  scale_y_log10() +  
  scale_x_continuous(breaks = seq(0, 90, 10))  
  
M = ggplot(data, aes(x = age, y = nMx, color = Country, group = Country)) +  
  geom_line() + geom_point() +  
  labs(title = "Age-Specific Mortality Rates", x = "Age", y = "nMx") +  
  theme_minimal() +  
  theme(legend.position = "right", legend.direction = "horizontal", legend.title = element_blank()) +  
  scale_x_continuous(breaks = seq(0, 90, 10))  
  
plot(lnM)
```



```
plot(M)
```



b. Compute the (unstandardised) crude death rates, CDR, for both countries and discuss the results.

Hint: Crude Death Rates can be calculated by dividing the total number of deaths by the total number of people. As a convention, death rates could be reported in thousands. You can use the R function `colSums(nDx)` to calculate the total number of deaths for each country, considering each country is represented by one row of ${}_nD_x$.

```
CDR = colSums(nDx)/colSums(nNx)*1000
sprintf("CDR in %s is %.2f deaths per 1,000", countries, CDR)

## [1] "CDR in Kazakhstan is 7.42 deaths per 1,000"
## [2] "CDR in Sweden is 10.55 deaths per 1,000"
```

c. Compute the standardised death rates for both countries (use the average age distribution of the two countries as the standard); and discuss the results.

The average age distribution can be computed in the following manner: (i) compute the relative age distribution for each country (i.e., the proportion of the population in each age group); and (ii) take the average of the two proportions for each age group.

Hint: The standard is calculated as the average of W , and W has been previously calculated. You can use the R function `rowSums(W)` to calculate the sum by rows and then divide by the number of countries. Finally, standardised rates can be calculated as the weighted average of the age-specific mortality rates.

```
standard = rowSums(W)/length(countries)
SDR = colSums(nMx*standard)*1000
sprintf("the SDR is %s: %.2f deaths per 1,000", countries, SDR)

## [1] "the SDR is Kazakhstan: 11.88 deaths per 1,000"
## [2] "the SDR is Sweden: 7.37 deaths per 1,000"
```

d. How would the standardised rates be different if we had used the Swedish age distribution as the standard? What if we had used the Kazakh age distribution?

Hint: You can repeat the previous step, using as the standard the column of W specific to each country.

```
SDR_Sweden      = colSums(nMx*W[, "Sweden"])*1000
SDR_Kazakhstan  = colSums(nMx*W[, "Kazakhstan"])*1000
tABLe           = t(data.frame(CDR, SDR, SDR_Sweden, SDR_Kazakhstan))
print(tABLe)
```

```
##           Kazakhstan      Sweden
## CDR           7.423042 10.547561
## SDR           11.881997  7.374094
## SDR_Sweden     16.340952 10.547561
## SDR_Kazakhstan  7.423042  4.200627
```

Exercise 2: Indirect standardisation

Let's assume that we didn't know the age distribution of deaths for Kazakhstan, but that we had an estimate of the total number of deaths 64,572, and the age distribution of the population.

In such circumstances, we can no longer conduct direct age-standardisation but can still compare the mortality regime in the two populations via indirect standardisation. Compute the Comparative Mortality Ratio, CMR (sometimes also referred to as the Standardised Mortality Ratio, SMR), and interpret the result.

Hint: You can use the function `sum(nDx)` to calculate the “observed number of deaths” in Kazakhstan (i.e., the second column of ${}_nD_x$). To calculate the “expected number of deaths”, assuming Kazakhstan had Sweden's age-specific mortality rates (${}_nM_x$), compute the sum of the product of Kazakhstan's ${}_nN_x$ and Sweden's ${}_nM_x$.

```
SMR              = sum(nDx[, "Kazakhstan"])/sum(nMx[, "Sweden"]*nNx[, "Kazakhstan"])
sprintf("SMR is %.2f", SMR)
```

```
## [1] "SMR is 1.77"
```

Optional steps to save your results

You may want to save your outputs at a specific location on your computer, and the first step is to identify your default working directory, using the function `getwd()`, as shown below:

```
getwd()
```

```
## [1] "/Users/lshjr3/Documents/Demographic_Methods/Practical_2"
```

You can change the working directory using the function `setwd(path)`, providing a new path to the folder, for example:

```
setwd("~/Documents/Demographic_Methods/Practical_2")
getwd()
```

```
## [1] "/Users/lshjr3/Documents/Demographic_Methods/Practical_2"
```

Then, you can save the resulting population pyramid to that specified location using `ggsave()`, just providing specif width and height of the figure in inches.

```
ggsave("lnM.png", plot = lnM, width = 5, height = 4)
ggsave("M.png", plot = M, width = 5, height = 4)
```

Finally, you may want to follow the best practice of saving files in a dedicated folder rather than in the default working directory. This approach is often more convenient because data and output files can be organized in separate locations, while the working directory can be reserved for other purposes. To do so, you can define the path to a specific folder and use it when saving outputs. You can use the function `paste0(string_1, ..., string_n)` to concatenate multiple characters and numbers without adding any spaces or separators between them, and combine the path and the name of the plot as shown:

```
pAth          = "~/Documents/Demographic_Methods/Practical_2/"
ggsave(paste0(pAth, "lnM.png"), plot = lnM, width = 5, height = 4)
ggsave(paste0(pAth, "M.png"), plot = M, width = 5, height = 4)
```